

Relatório de Auditoria de Segurança — Editorial Architect

Avaliação Estática e Dinâmica de Vulnerabilidades em Código-Fonte, Controle de Acesso e Gestão de Segredos

Projeto Auditado:	Editorial Architect (Fullstack Platform)	Data da Auditoria:	30 de Agosto de 2026
Escopo Repositórios:	editorial-backend + editorial-frontend	Classificação:	RESTRITO / CONFIDENCIAL
Stack Backend:	Node.js 18+, Express 4.18, Prisma ORM 6.19 (MySQL)	Stack Frontend:	React 19, Vite 6, Tailwind CSS, CASL Ability
Auth & Gateway:	JWT + Bcrypt (Backend), Asaas API v3, Gemini AI	Deploy & Infra:	Railway Cloud (railway.toml / railway.json)

1. NOTA METODOLÓGICA & DETECÇÃO DE STACK

A presente auditoria adotou o modelo de análise *White-Box* sobre 100% da base de código dos repositórios editorial-backend e editorial-frontend. Antes do início dos testes, a arquitetura e a cadeia de suprimentos de software foram mapeadas para calibrar com precisão os vetores de ataque equivalentes na stack real:

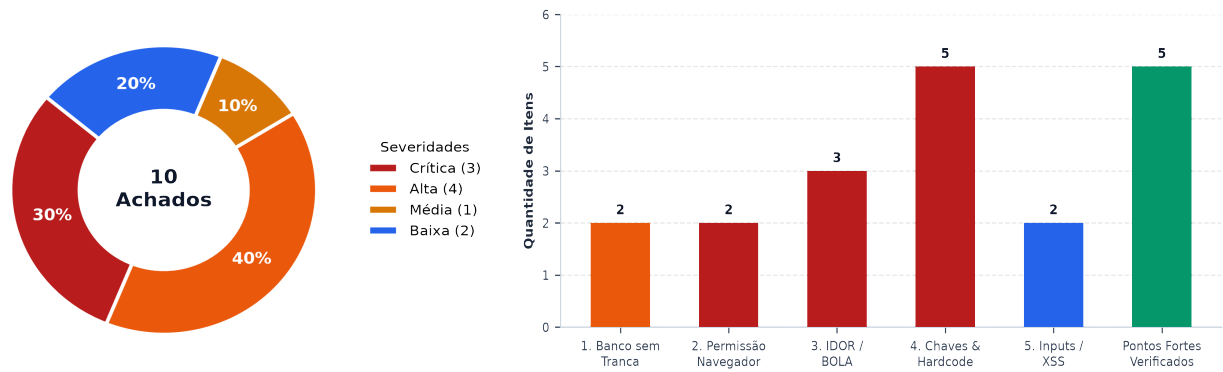
Categoria Auditada	Mapeamento Técnico na Stack do Projeto	Mecanismo do Projeto & Cobertura
1. Banco sem Tranca (Isolamento Tenancy)	Stack utiliza MySQL via Prisma ORM sem Row-Level Security (RLS) nativo. A tranca depende de queries com where: { userId } em cada handler.	Auditado handler por handler em busca de queries com findMany, findFirst e agregações sem filtro do usuário da sessão.
2. Permissão no Navegador (Bypass de RBAC)	Frontend implementa controle visual com @casl/ability e rotas com ProtectedRoute requireAdmin. Backend usa checkPermission.	Cruzamento exato entre os gates de UI (menus, botões de admin, páginas restritas) e os endpoints correspondentes no Express.
3. IDOR / BOLA (Broken Object Level)	Operações CRUD que recebem IDs (ULID ou Asaas ID) via req.params ou req.body.	Inspeção exaustiva em todos os endpoints que aceitam identificadores para confirmar validação de propriedade.
4. Chaves Expostas (Hardcode & Defaults)	Tokens JWT, chaves de API (Gemini, Asaas), webhooks e credenciais de banco em código, .env, seed.ts e histórico git.	Varredura do histórico git completo (commits anteriores), inspeção do bundler Vite e análise de fallbacks inseguros.
5. Inputs / XSS (Sanitização e Injeção)	Frontend React 19 + Tailwind; Backend Express com res.json(). Busca por dangerouslySetInnerHTML, innerHTML, eval().	Varredura em todos os 26 componentes e análise das funções de sanitização de strings no backend.

2. RESUMO EXECUTIVO & PANORAMA DE RISCO

A auditoria de segurança identificou um total de **10 achados de segurança** e **5 pontos fortes arquiteturais verificados**. Entre os achados, destacam-se **3 vulnerabilidades de severidade CRÍTICA** que permitem tomada completa de contas (Account Takeover), forja de sessões administrativas e redefinição não autorizada de senhas de qualquer usuário cadastrado no sistema.

Severidade	Total	Impacto Sistêmico	SLA de Resolução Recomendado
CRÍTICA	3	Comprometimento total da base de usuários, tomada de contas mestre e bypass global de autenticação.	Imediato (24 horas)
ALTA	4	Vazamento de chaves no bundle público, IDOR em transações financeiras e credenciais padrão em seed.	Até 3 dias úteis
MÉDIA	1	Bypass de regras comerciais e limites de plano no backend (white-label desprotegido).	Até 10 dias úteis
BAIXA / INFO	2	Sanitização incompleta de strings e ausência de validação rígida de protocolo em URLs.	Próximo ciclo de release
PONTOS FORTES	5	Isolamento estrito no módulo de Projetos, RBAC no Express para rotas admin e ausência total de XSS.	Manter padrão e monitorar

Distribuição de Severidades e Cobertura por Categoria



Diagnóstico de Risco Central da Plataforma:

Embora o módulo nuclear de manipulação de projetos (project.controller.ts) tenha sido implementado com alta maturidade e isolamento estrito de inquilinos (verificando where: { userId: req.user.id } em todos os pontos), as adições recentes nos fluxos de pagamento (payment.controller.ts) e autenticação simplificada (auth.controller.ts) introduziram atalhos de desenvolvimento severos. A funcionalidade de *auto-login* no checkout e o reset de senha unificado não exigem credenciais anteriores nem tokens criptográficos temporários, permitindo a apropriação arbitrária de contas e dados cadastrais.

3. PONTOS FORTES & PONTOS FRACOS SISTÊMICOS

A segurança da plataforma apresenta um forte contraste entre áreas com blindagem exemplar e fluxos periféricos vulneráveis. Abaixo estão documentadas as defesas efetivas comprovadas em código e os pontos cegos estruturais:

✓ PONTOS FORTES AUDITADOS (EVIDÊNCIAS DE CÓDIGO CORRETO)

Componente / Recurso	Evidência Verificada no Código Real	Garantia de Segurança Obtida
Isolamento Estrito em Projetos project.controller.ts	Linhas 48, 69, 98, 135 e 165: Todos os métodos (getProjects, getProjectById, updateProject, deleteProject e getDashboardStats) aplicam where: { userId: req.user.id }.	Impossibilita IDOR ou vazamento horizontal de projetos entre usuários.
Validação de RBAC no Servidor server.ts:100-115	Todas as rotas sob /api/admin/* (roles, permissions, users, analytics) aplicam checkPermission(action, subject) antes dos controladores.	Usuários comuns com token JWT válido recebem HTTP 403 caso tentem acessar rotas de administração.
Imunidade a XSS na UI 26 componentes React	Zero instâncias de dangerouslySetInnerHTML, innerHTML ou v-html. Textos gerados por IA são interpolados via nós de texto React padrão.	Defesa nativa contra Stored e Reflected XSS em toda a interface do usuário.
Validação de Posse no Perfil auth.controller.ts:213	Em updateProfile e getUserAnalytics, o ID alvo é extraído exclusivamente de req.user.id, sem aceitar IDs no body.	Garante que nenhum usuário consiga editar o perfil de terceiros via rota regular.
Proteção contra Exclusão de Roles de Sistema admin.controller.ts:143	Validação explícita if (role.isSystem) return res.status(400) bloqueia deleção de perfis essenciais como 'Admin System'.	Evita negação de serviço e bloqueio permanente do console administrativo.

X PONTOS FRACOS CENTRAIS (VULNERABILIDADES SISTÊMICAS)

Ponto Cego Arquitetural	Mecanismo da Falha	Consequência Crítica
Account Takeover no Checkout subscribePlan	Endpoint busca usuário existente apenas por e-mail fornecido no body sem exigir senha nem token de sessão, emitindo novo JWT.	Qualquer pessoa assume controle de contas existentes informando o e-mail no checkout.
Redefinição Arbitrária de Senha resetPassword	Rota pública altera hash da senha diretamente por e-mail sem envio de token OTP/link por e-mail e devolve JWT autenticado.	Sequestro instantâneo da conta editor@elite.com (Admin) ou de clientes.
Segredo JWT Público por Fallback auth.middleware.ts:7	Código utiliza 'fallback_secret' se JWT_SECRET não existir, e a chave esteve commitada no git.	Forja de tokens arbitrários com privilégio Admin System.
Vazamento de Chave no Vite vite.config.ts:28-32	Vite compila process.env.API_KEY com a chave real da Gemini no bundle estático do frontend.	Chave de API do Google Gemini acessível publicamente a qualquer visitante do site.
IDOR em Transações Pix getPaymentStatus	Rota de consulta de pagamento não valida posse de usuário e aceita chamadas não autenticadas.	Exposição de transações financeiras, QR Codes Pix e datas de pagamento.

4. TABELA CONSOLIDADA DE ACHADOS POR CATEGORIA

A tabela a seguir consolida todos os achados com seu arquivo, linha exata, severidade ponderada e descrição técnica do vetor de exploração:

Sev.	Cat.	Arquivo : Linhas	Vulnerabilidade / Descrição Técnica do Achado
CRÍT	1 e 3	backend/src/controllers/payment.controller.ts:101-106, 283-287	BOLA / Account Takeover via Checkout: O endpoint assume usuário existente via email sem auth e emite JWT da vítima no response.
CRÍT	2 e 3	backend/src/controllers/auth.controller.ts:380-417	Redefinição Arbitrária de Senhas: POST /api/auth/reset-password altera senha de qualquer usuário por e-mail sem verificação e devolve JWT.
CRÍT	4	backend/src/middleware/auth.middleware.ts:7 backend/src/controllers/auth.controller.ts:11	Segredo JWT com Fallback Inseguro e Chave Exposta no Git: Fallback 'fallback_secret' e histórico git contém editorial_architect_secret_key_2026.
ALTA	1 e 3	backend/src/controllers/payment.controller.ts:322-396	IDOR em Status de Pagamentos: GET /api/payments/:id/status aceita qualquer ID sem autenticação nem filtro por userId, vazando dados Pix.
ALTA	4	frontend/vite.config.ts:18, 28-32	Exposição de Chave de API Gemini no Bundle JS do Vite: Configuração define: 'process.env.API_KEY' baka segredo no código cliente.
ALTA	4	backend/prisma/seed.ts:208-227	Credencial Padrão de Super Administrador: População de banco cria conta editor@elite.com com senha 'senha123' e role Admin System.
ALTA	4	backend/src/controllers/payment.controller.ts:472	Bypass de Autenticação em Webhook Asaas: Condição configuredToken && webhookToken !== configuredToken passa sem auth se variável for nula.
MÉD	2	backend/src/controllers/project.controller.ts:6-43, 90-126	Bypass de Restrições Comerciais de Plano (White-Label): Backend não valida flag hasWhiteLabel ao gravar companyName no projeto.
BAIX	5	backend/src/lib/validators.ts:44-47	Sanitização Incompleta de Strings: Método sanitizeString apenas executa .trim() sem higienizar caracteres especiais HTML.
BAIX	5	frontend/components/DocumentPreview.tsx:180-188	Ausência de Validação de Protocolo em Watermark: Prop watermarkImage aceita URIs arbitrárias sem restrição de protocolo ou MIME type.

Detalhamento Técnico dos Achados Críticos (Severidade Máxima):

[CRÍTICA] SEC-01: Tomada de Conta de Qualquer Usuário via Checkout (BOLA / Broken Auth)

Arquivo: editorial-backend/src/controllers/payment.controller.ts:101-106, 283-287

```
if (!user && userEmail) { user = await prisma.user.findUnique({ where: { email: userEmail } }); } ... const userToken = jwt.sign({ id: user.id, email: user.email }, JWT_SECRET, { expiresIn: '7d' }); return res.json({ token: userToken, ... });
```

Por que é explorável: O endpoint POST /api/payments/subscribe utiliza optionalAuthenticateToken. Se um atacante não autenticado enviar qualquer requisição de assinatura contendo o e-mail de uma vítima (incluindo o e-mail do administrador editor@elite.com), o backend localiza o usuário existente, sobrescreve seus dados cadastrais (telefone, CPF) e gera um token JWT assinado válido por 7 dias, devolvendo-o diretamente no corpo da resposta HTTP. O atacante ganha acesso completo imediato à conta.

[CRÍTICA] SEC-02: Redefinição Arbitrária de Senha sem Autenticação (Account Takeover)

Arquivo: editorial-backend/src/controllers/auth.controller.ts:380-417

```
const user = await prisma.user.findUnique({ where: { email: String(email).trim() } }); ... const senhaHash = await bcrypt.hash(novaSenha, SALT_ROUNDS); await prisma.user.update({ where: { id: user.id }, data: { passwordHash: senhaHash } }); const token = jwt.sign({ id: user.id, email: user.email }, JWT_SECRET); return res.json({ token });
```

Por que é explorável: A rota POST /api/auth/reset-password é totalmente pública. Ela não exige a senha anterior, não envia código OTP para o e-mail e não valida nenhum token temporário assinado. Qualquer atacante que saiba o e-mail de um usuário pode redefinir sua senha para um valor arbitrário e receber um token JWT na mesma requisição para autenticar imediatamente.

[CRÍTICA] SEC-03: Segredo JWT Fallback Hardcoded e Exposição Prévia no Histórico Git

Arquivo: editorial-backend/src/middleware/auth.middleware.ts:7 | Commit 2bdfcf8

```
const JWT_SECRET = process.env.JWT_SECRET || 'fallback_secret';
```

Por que é explorável: Se a variável JWT_SECRET for omitida no ambiente de produção (ou se falhar ao carregar via dotenv), o servidor assume silenciosamente 'fallback_secret'. Um atacante pode forjar tokens JWT contendo { id: 'qualquer-id', role: 'Admin System' }. Além disso, no commit 2bdfcf8f98164f1a8e9444fae18aaba26e20b807^, o arquivo .env com editorial_architect_secret_key_2026 foi versionado e permanece no histórico do repositório Git.

Detalhamento Técnico dos Achados de Severidade Alta e Média:

[ALTA] SEC-04: IDOR e Vazamento de Dados Financeiros em Status de Pagamento

Arquivo: editorial-backend/src/controllers/payment.controller.ts:322-396

```
const paymentId = String(req.params.id); let transaction = await prisma.paymentTransaction.findUnique({ where: { asaasPaymentId: paymentId } }); ... return res.json({ id: paymentId, status, pixQrCodeUrl, pixCopyPaste, ... });
```

Por que é explorável: A rota GET /api/payments/:id/status utiliza optionalAuthenticateToken e aceita qualquer paymentId (Asaas ID) sem validar se a transação pertence ao usuário da requisição (transaction.userId !== req.user?.id). Permite inspecionar status de pagamento, links de QR Code Pix e chaves copia-e-cola de qualquer cliente. Além disso, força polling e atualização de plano.

[ALTA] SEC-05: Exposição de Chave da API Google Gemini no Bundle Público do Vite

Arquivo: editorial-frontend/vite.config.ts:18, 28-32

```
const apiKey = env.VITE_GEMINI_API_KEY || env.API_KEY || ''; ... define: { 'process.env.API_KEY': JSON.stringify(apiKey) }
```

Por que é explorável: A diretiva define do Vite substitui em tempo de compilação qualquer menção a process.env.API_KEY pelo valor em texto claro da chave contida no arquivo .env.local. Ao compilar com pnpm build, a chave de produção do Gemini é embutida nos arquivos JavaScript distribuídos publicamente no diretório dist/assets/.

[ALTA] SEC-06: Credencial Padrão de Super Administrador Criada no Seed

Arquivo: editorial-backend/prisma/seed.ts:208-227

```
const emailPadrao = 'editor@elite.com'; const senhaHash = await bcrypt.hash('senha123', 10); await prisma.user.create({ data: { email: emailPadrao, passwordHash: senhaHash, roleId: dbRoles['Admin System'].id } });
```

Por que é explorável: O script de seed popula uma conta de Super Administrador (Admin System) com a senha estática previsível 'senha123'. Ambientes de homologação, staging ou produção onde o seed for executado ficam imediatamente vulneráveis a login administrativo não autorizado.

[ALTA] SEC-07: Bypass da Validação de Token de Webhook do Asaas por Variável Nula

Arquivo: editorial-backend/src/controllers/payment.controller.ts:472

```
if (configuredToken && webhookToken !== configuredToken) { return res.status(401).json({ error: 'Token de webhook inválido' }); }
```

Por que é explorável: Se a variável ASAAS_WEBHOOK_TOKEN não for preenchida no ambiente do servidor, configuredToken avalia para undefined (falso). A condição de segurança é ignorada e qualquer atacante externo pode enviar notificações falsas de PAYMENT_CONFIRMED para /api/webhooks/asaas, ativando planos pagos sem nenhum desembolso financeiro.

5. RECOMENDAÇÕES PRIORIZADAS & PLANO DE REMEDIAÇÃO

As medidas de correção foram divididas em 3 níveis de prioridade com base na severidade e na probabilidade de exploração remota:

Prioridade	Ações Técnicas Imediatas	Arquivos Alvo & Correção
P1 — URGENTE (24h - 48h)	1. Corrigir Account Takeover no Checkout: Se o e-mail já pertencer a outro usuário e a requisição não tiver token do mesmo, rejeitar com 409 Conflict ou exigir login prévio. 2. Blindar o Reset de Senha: Implementar fluxo com envio de token criptográfico temporário por e-mail (válido por 15 min) com hash salvo no banco. 3. Remover Fallbacks de JWT: Adicionar validação no startup do servidor: rejeitar inicialização se JWT_SECRET não estiver definido ou for fraco. 4. Rotacionar Segredo JWT e Chaves: Rotacionar o JWT_SECRET e a chave do Gemini devido à exposição no Git e no Vite.	payment.controller.ts:101 auth.controller.ts:380 auth.middleware.ts:7 server.ts:40
P2 — CURTO PRAZO (1 semana)	5. Corrigir IDOR de Pagamento: Exigir autenticação obrigatória em /api/payments/:id/status e validar where: { asaasPaymentId: id, userId: req.user.id }. 6. Limpar vite.config.ts: Remover injeção de process.env.API_KEY no frontend, mantendo as chamadas à IA restritas ao backend. 7. Remover Credenciais Padrão do Seed: Gerar senha aleatória segura no seed ou ler de variáveis de ambiente para inicialização de administradores. 8. Travar Webhook sem Token: Rejeitar webhooks se ASAAS_WEBHOOK_TOKEN não estiver configurado no servidor (falha segura por padrão).	payment.controller.ts:322 vite.config.ts:28 prisma/seed.ts:214 payment.controller.ts:472
P3 — MÉDIO PRAZO (2 semanas)	9. Validação de Limites de Plano no Backend: Criar validação em project.controller.ts para impedir custom branding (white-label) se o plano do usuário não permitir. 10. Sanitização e Validação Rígida: Utilizar biblioteca de sanitização/validação (Zod / DOMPurify) e validar esquemas de URLs para imagens personalizadas.	rbac.middleware.ts validators.ts DocumentPreview.tsx

6. ISSUES PARA O GITHUB (TEXTO COMPLETO PRONTO PARA COPIAR E COLAR)

Abaixo estão os templates completos em formato Markdown de cada issue acionável identificada na auditoria. Cada bloco está delimitado individualmente entre --- ISSUE n --- e --- FIM ISSUE n ---.

Issue #1 — [Segurança] [Crítica] Account Takeover no Checkout de Assinaturas (subscribePlan)

```
--- ISSUE 1 ---
### [Segurança] [Crítica] Account Takeover no Checkout de Assinaturas (subscribePlan)
**Labels sugeridas:** `security`, `severity: critical`, `backend`

#### Descrição do Problema
O endpoint `POST /api/payments/subscribe` (`backend/src/controllers/payment.controller.ts`) permite que uma requisição não autenticada envie qualquer e-mail no corpo da requisição (`req.body.email`). Se o e-mail já existir no banco de dados, o backend assume o registro da vítima sem validar sessão ou credenciais, atualiza dados cadastrais e assina um token JWT em nome da vítima, retornando-o no response HTTP.

#### Evidência
- **Arquivo:** `editorial-backend/src/controllers/payment.controller.ts` (Linhas 101-106 e 283-287)
  ```typescript
 if (!user && userEmail) {
 user = await prisma.user.findUnique({
 where: { email: userEmail },
 include: { plan: true }
 });
 }
 // ...
 const userToken = jwt.sign(
 { id: user.id, email: user.email },
 JWT_SECRET,
 { expiresIn: '7d' }
);
 return res.json({ message: '...', token: userToken });
  ```

#### Impacto
Tomada total de conta (Account Takeover) de qualquer usuário da plataforma, incluindo o Super Admin (`editor@elite.com`), apenas enviando seu e-mail no formulário de checkout.

#### Sugestão de Correção
Se o e-mail informado já existir no banco de dados:
1. Se a requisição não possuir um token JWT válido pertencente àquele usuário, rejeitar a requisição com HTTP `409 Conflict` e instruir o usuário a fazer login antes de alterar sua assinatura.
2. Nunca emitir tokens JWT para requisições anônimas em contas pré-existentes.

#### Critérios de Aceite
- [ ] Requisições para `/api/payments/subscribe` com e-mail já cadastrado sem autenticação retornam HTTP 409.
- [ ] Usuários autenticados só podem assinar planos para o seu próprio `req.user.id`.
- [ ] Testes automatizados cobrindo tentativa de subscrição anônima em e-mail de terceiro.
--- FIM ISSUE 1 ---
```


Issue #2 — [Segurança] [Crítica] Redefinição Arbitrária de Senha sem Autenticação (resetPassword)

```
--- ISSUE 2 ---
### [Segurança] [Crítica] Redefinição Arbitrária de Senha sem Autenticação (resetPassword)
**Labels sugeridas:** `security`, `severity: critical`, `backend`, `auth`

#### Descrição do Problema
O endpoint público `POST /api/auth/reset-password` (`backend/src/controllers/auth.controller.ts`) recebe `email` e `novaSenha`, localiza o usuário diretamente pelo e-mail e substitui o hash da senha imediatamente no banco, gerando em seguida um token de sessão JWT e retornando-o ao chamador. Não há envio de link por e-mail, nem código OTP, nem validação de senha anterior.

#### Evidência
- **Arquivo:** `editorial-backend/src/controllers/auth.controller.ts` (Linhas 392-408)
  ```typescript
 const user = await prisma.user.findUnique({ where: { email: String(email).trim() } });
 // ...
 const senhaHash = await bcrypt.hash(novaSenha, SALT_ROUNDS);
 await prisma.user.update({
 where: { id: user.id },
 data: { passwordHash: senhaHash }
 });
 const token = jwt.sign({ id: user.id, email: user.email }, JWT_SECRET);
 return res.json({ message: 'Senha atualizada com sucesso!', token });
  ```
- **Impacto**
  Qualquer visitante da internet pode alterar a senha de qualquer conta do sistema e autenticar-se imediatamente.

- **Sugestão de Correção**
  Implementar fluxo seguro de redefinição de senha em duas etapas:
  1. Endpoint `forgot-password` gera um token aleatório criptograficamente seguro (`crypto.randomBytes`), salva o hash com expiração (15 min) e envia link único por e-mail.
  2. Endpoint `reset-password` exige o token recebido por e-mail para permitir o cadastro da nova senha.

- **Critérios de Aceite**
  - [ ] Endpoint não aceita troca direta de senha sem token de redefinição validado.
  - [ ] Tokens de redefinição possuem expiração máxima de 15 minutos e uso único.
  - [ ] Não é possível autenticar nem receber JWT sem validação de e-mail.
  --- FIM ISSUE 2 ---
```

Issue #3 — [Segurança] [Crítica] Segredo JWT Hardcoded como Fallback e Chave Exposta no Git

```
--- ISSUE 3 ---
### [Segurança] [Crítica] Segredo JWT Hardcoded como Fallback e Chave Exposta no Git
**Labels sugeridas:** `security`, `severity: critical`, `backend`, `config`

#### Descrição do Problema
Em múltiplos arquivos (`auth.middleware.ts`, `auth.controller.ts`, `payment.controller.ts`), a constante `JWT_SECRET` utiliza a string estática `fallback_secret` caso a variável de ambiente não esteja presente. Além disso, a chave `editorial_architect_secret_key_2026` esteve commitada no histórico do repositório Git (commit `2bdfcf8`).

#### Evidência
- **Arquivo:** `editorial-backend/src/middleware/auth.middleware.ts` (Linha 7)
  ```typescript
 const JWT_SECRET = process.env.JWT_SECRET || 'fallback_secret';
  ```
- **Git Commit:** `2bdfcf8f98164f1a8e9444fae18aaba26e20b807^:.env`

- **Impacto**
  Possibilidade de forja de tokens JWT válidos com perfil `Admin System` ou qualquer identidade por atacantes externos que conheçam o segredo público.

- **Sugestão de Correção**
  1. Adicionar validação estrita na inicialização do servidor (`server.ts`): abortar a inicialização caso `process.env.JWT_SECRET` esteja ausente ou seja menor que 32 caracteres.
  2. Remover fallbacks inseguros (`|| 'fallback_secret'`).
  3. Rotacionar imediatamente o segredo de produção nas variáveis de ambiente da Railway.

- **Critérios de Aceite**
  - [ ] A aplicação encerra o processo (`process.exit(1)`) se `JWT_SECRET` for nulo ou inválido.
  - [ ] O segredo antigo foi revogado e substituído em produção por string de alta entropia.
  - [ ] Nenhuma menção a `fallback_secret` em código-fonte.
  --- FIM ISSUE 3 ---
```

Issue #4 — [Segurança] [Alta] IDOR e Vazamento de Dados Pix em Polling de Pagamentos

```
--- ISSUE 4 ---
### [Segurança] [Alta] IDOR e Vazamento de Dados Pix em Polling de Pagamentos
**Labels sugeridas:** `security`, `severity: high`, `backend`, `idor`

#### Descrição do Problema
A rota `GET /api/payments/:id/status` aceita chamadas não autenticadas ou com qualquer token e busca a transação diretamente pelo ID do gateway (`asaasPaymentId`) sem checar se a transação pertence ao `req.user.id`.

#### Evidência
- **Arquivo:** `editorial-backend/src/controllers/payment.controller.ts` (Linhas 324-334)
  ```typescript
 const paymentId = String(req.params.id);
 let transaction = await prisma.paymentTransaction.findUnique({
 where: { asaasPaymentId: paymentId }
 });
 // Retorna pixQrCodeUrl, pixCopyPaste, status e paymentDate sem checar transaction.userId
  ```
- **Impacto**
  Vazamento de dados financeiros de outros assinantes, código Cópia e Cola Pix e possibilidade de forçar polling e alteração de status no banco de dados.

- **Sugestão de Correção**
  1. Exigir autenticação via `authenticateToken`.
  2. Validar que a transação pertence ao usuário autenticado: `transaction.userId === req.user.id` (ou perfil com permissão de leitura de finanças/admin).

- **Critérios de Aceite**
  - [ ] A rota rejeita requisições não autenticadas com HTTP 401.
  - [ ] Usuários autenticados recebem HTTP 404/403 ao consultar transações de outros clientes.
  --- FIM ISSUE 4 ---
```

Issue #5 — [Segurança] [Alta] Exposição da Chave Gemini no Bundle do Frontend via Vite

```
--- ISSUE 5 ---
### [Segurança] [Alta] Exposição da Chave Gemini no Bundle do Frontend via Vite
**Labels sugeridas:** `security`, `severity: high`, `frontend`, `secrets`

#### Descrição do Problema
O arquivo de configuração `vite.config.ts` utiliza a diretiva `define` para embutir a variável `process.env.API_KEY` diretamente nos artefatos compilados do cliente, expondo a chave de API do Gemini no JavaScript público da SPA.

#### Evidência
- **Arquivo:** `editorial-frontend/vite.config.ts` (Linhas 18 e 28-32)
  ```typescript
 const apiKey = env.VITE_GEMINI_API_KEY || env.API_KEY || "";
 // ...
 define: {
 'process.env.API_KEY': JSON.stringify(apiKey),
 'process.env': {
 NODE_ENV: JSON.stringify(mode),
 API_KEY: JSON.stringify(apiKey)
 }
 }
  ```
- **Impacto**
  Qualquer visitante pode inspecionar os arquivos JS no navegador e extrair a chave de API do Gemini, consumindo cotas ou gerando custos financeiros indevidos.

- **Sugestão de Correção**
  1. Remover a injeção de `API_KEY` do `vite.config.ts`.
  2. Manter todas as chamadas de IA restritas ao endpoint backend `/api/ai/generate`.
  3. Rotacionar a chave do Gemini no painel do Google AI Studio.

- **Critérios de Aceite**
  - [ ] O bundle em `dist/assets/*.js` não contém chaves de API nem menções a chaves do Gemini.
  - [ ] Chave antiga revogada no Google Cloud / AI Studio.
  --- FIM ISSUE 5 ---
```

Issue #6 — [Segurança] [Alta] Credencial Previsível de Administrador no Seed e Falha de Validação em Webhook

```
--- ISSUE 6 ---
### [Segurança] [Alta] Credencial Previsível de Administrador no Seed e Falha de Validação em Webhook
**Labels sugeridas:** `security`, `severity: high`, `backend`, `config`

#### Descrição do Problema
1. O seed do Prisma cria a conta `editor@elite.com` com senha `senha123` e permissões de Super Admin (`Admin System`).
2. O controlador de webhook (`payment.controller.ts`) possui uma condição de verificação de token que não bloqueia requisições se a variável `ASAAS_WEBHOOK_TOKEN` for nula.

#### Evidência
- **Arquivo:** `editorial-backend/prisma/seed.ts` (Linhas 214-226)
- **Arquivo:** `editorial-backend/src/controllers/payment.controller.ts` (Linha 472)
``typescript
if (configuredToken && webhookToken !== configuredToken) { ... }
...

#### Impacto
Comprometimento administrativo em novas instalações e possibilidade de falsificação de pagamentos para liberação ilimitada de planos se o webhook token não estiver preenchido.

#### Sugestão de Correção
1. Modificar `seed.ts` para ler a senha de `process.env.INITIAL_ADMIN_PASSWORD` ou gerar senha aleatória exibida apenas no console durante o setup inicial.
2. Exigir explicitamente que `configuredToken` exista e rejeitar requisições de webhook com 500/401 caso não esteja configurado:
``typescript
if (!configuredToken || webhookToken !== configuredToken) {
return res.status(401).json({ error: 'Webhook não autorizado' });
}
...

#### Critérios de Aceite
- [ ] A execução do seed não define credenciais estáticas óbvias.
- [ ] A rota de webhook rejeita chamadas se `ASAAS_WEBHOOK_TOKEN` não estiver configurado.
--- FIM ISSUE 6 ---
```